

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

GitHub Repository: Autoencoders

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

3. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

The labels are not required as reconstruction targets. The original image is the target:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where x is the input image and $\hat{x}$ is its reconstruction.

Observed Output

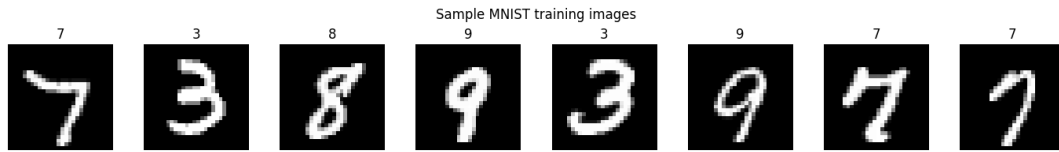


Figure 1: Sample images from the loaded MNIST training subset (10,000 training / 2,000 test images, normalized to $[0, 1]$).

4. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

Original image
Reconstructed image
Reconstruction error

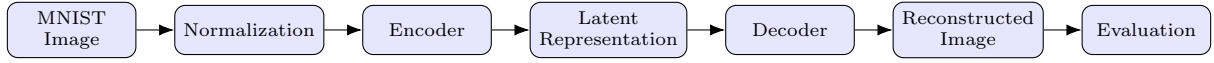
For the denoising model, display:

Original clean image
Noisy image
Denoised reconstruction

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

5. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.

The encoder maps the input to a lower-dimensional representation:

$$z = f_{\theta}(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_{\phi}(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$



The latent vector is a compressed representation of the input.

7. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.

Use the following architecture:



Use ReLU activation in hidden layers and sigmoid activation in the output layer.

Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

8. Required Analysis of the Basic Autoencoder

Plot 1: Original versus Reconstructed Images

Display at least 10 test images in a grid.

For every image, show:

Original | Reconstruction.

Students must comment on:

- preservation of digit shape;
- loss of fine details;
- blurred regions;
- differences between easy and difficult samples.

Plot 2: Training and Validation Reconstruction Loss

- X-axis: Epoch
- Y-axis: Reconstruction loss
- Curves: Training loss and validation loss

Required inference: Determine whether the model converges and whether there is evidence of overfitting.

Observed Output

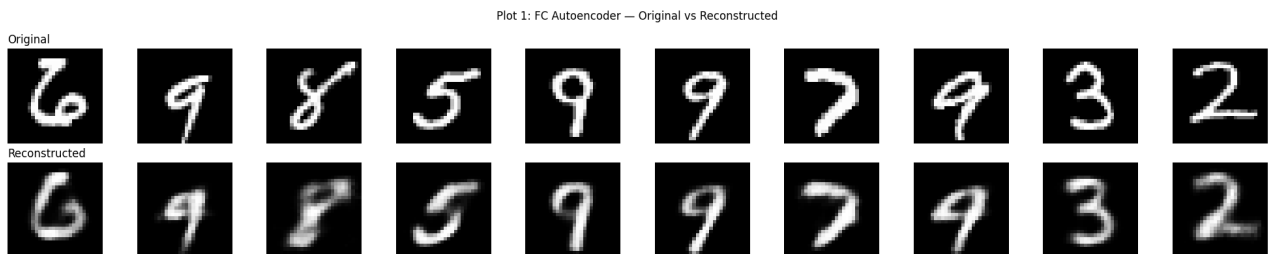


Figure 2: Plot 1 – Fully connected autoencoder: original (top) versus reconstructed (bottom) test digits.

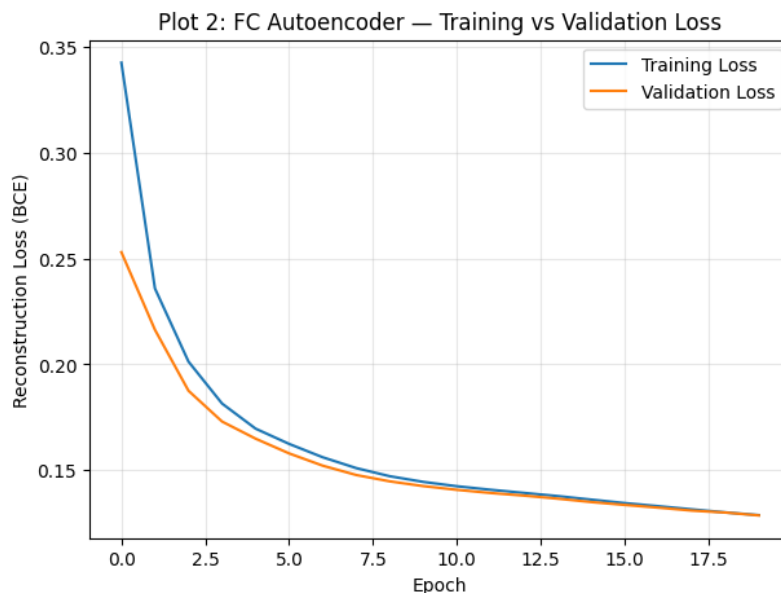


Figure 3: Plot 2 – Fully connected autoencoder: training and validation reconstruction loss over 20 epochs.

Inference: The overall digit shape is preserved for most samples, but the reconstructions are visibly blurred and fine strokes are smoothed out – for example the loop of the digit “8” is not cleanly separated and thin strokes in “9” and “7” are thickened. Simple, single-stroke digits such as “1”, “7” and “3” are reconstructed almost perfectly, while digits with loops or multiple crossing strokes (“8”, “5”) are the most difficult and show the largest visual distortion. In Plot 2, the training and validation loss curves decrease together and stay very close to each other throughout all

20 epochs (final training loss 0.1288 vs. validation loss 0.1286), indicating that the model converges smoothly with no evidence of overfitting.

9. Reconstruction Metrics

Students must calculate the following metrics on the test set.

Mean Squared Error

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i|.$$

Structural Similarity

Calculate SSIM between the original and reconstructed images.

The implementation may use:

<https://scikit-image.org/docs/stable/api/skimetrics.html>

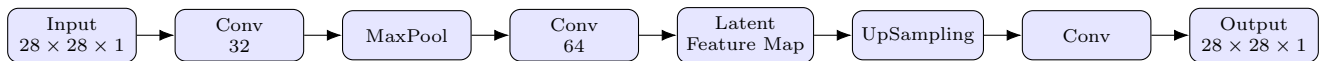
Report:

Metric	Value
Test MSE	0.022427
Test MAE	0.058893
Mean SSIM	0.734007

10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.

The conceptual architecture is:



A possible implementation is:

```

Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')
  
```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

Observed Output

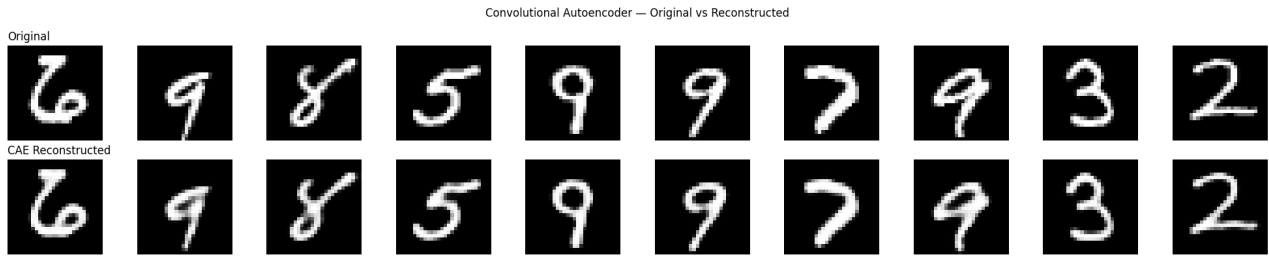


Figure 4: Convolutional autoencoder: original versus reconstructed test digits.

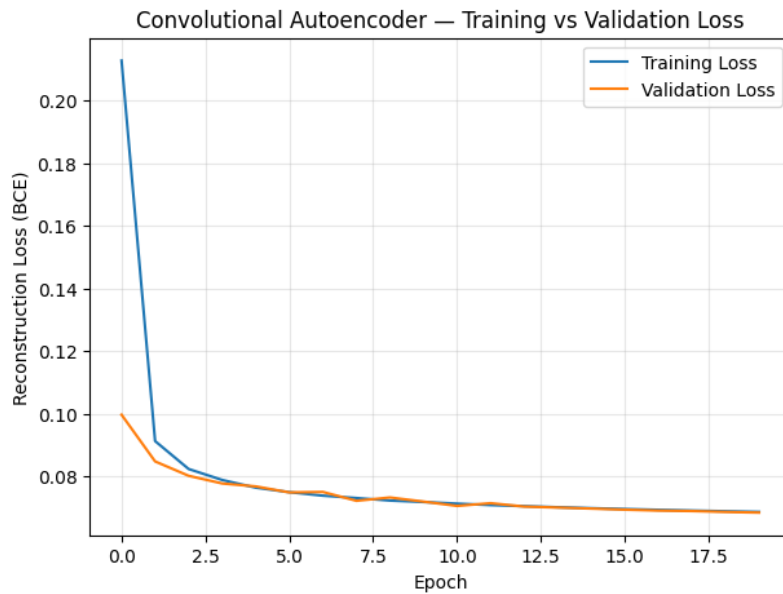


Figure 5: Convolutional autoencoder: training and validation reconstruction loss over 20 epochs (training time 23.4 s, 74,497 parameters).

The convolutional autoencoder converges to a much lower reconstruction loss than the fully connected model (final validation loss 0.0684 versus 0.1286) while using roughly one-third of the parameters, and the reconstructed digits are visually sharper with far less blurring around loops and edges.

11. Comparing Fully Connected and Convolutional Autoencoders

Train both models using the same training and test images.

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.022427	0.058893	0.734007	211,040
Convolutional AE	0.002496	0.014989	0.974557	74,497

Plot 3: Reconstruction Comparison

Display:

Original $\rightarrow$ FC-AE Reconstruction $\rightarrow$ CAE Reconstruction.

Required inference:

Discuss how preserving spatial locality through convolution affects image reconstruction.

Observed Output

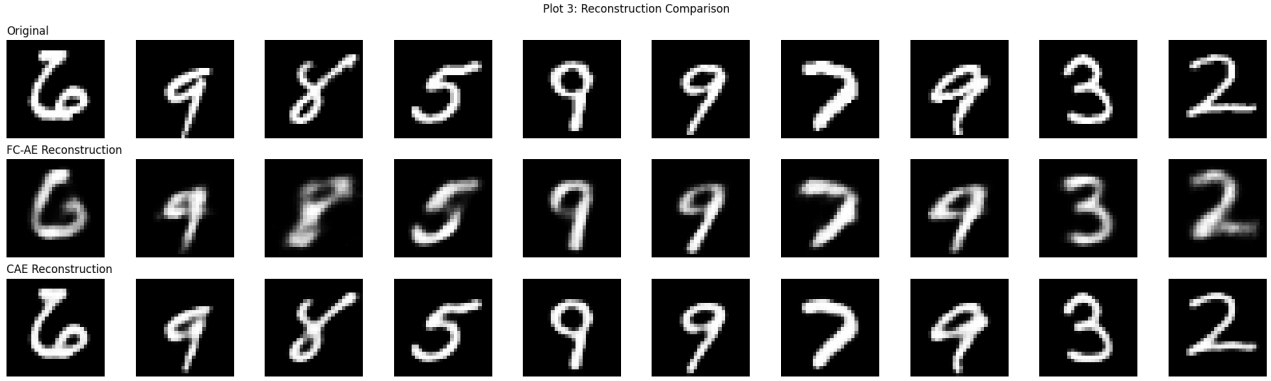


Figure 6: Plot 3 – Original image, FC-AE reconstruction, and CAE reconstruction, side by side.

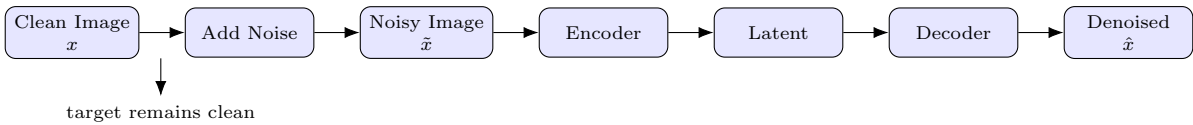
Inference: Preserving spatial locality through convolution and pooling lets the CAE exploit local pixel correlations that flattening destroys, so the CAE achieves a lower MSE (0.0025 vs. 0.0219) and MAE (0.0150 vs. 0.0589) and a much higher mean SSIM (0.975 vs. 0.734) than the fully connected autoencoder, despite having fewer parameters. Visually, the CAE reconstructions are sharper and preserve edges and loop structure (e.g. the “8” and “5”) far better than the FC-AE, which produces noticeably blurred outputs for the same digits.

12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image. The training mapping is:

$$\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where $\tilde{x}$ is a noisy version of the clean image x .



13. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.

Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

14. Required Denoising Plots

Plot 4: Clean vs. Noisy vs. Denoised

For at least 10 images, display:

Clean | Noisy | Denoised.

Plot 5: Noise Level vs. Reconstruction Quality

For each noise level, calculate:

MSE , MAE , $SSIM$.

Report:

Noise Level	MSE	MAE	SSIM
0.1	0.003511	0.017354	0.961892
0.2	0.004451	0.020670	0.948907
0.3	0.006691	0.027823	0.894538

Required inference:

Explain how increasing corruption affects reconstruction quality and identify whether the denoising model is able to recover the major structure of the original image.

Observed Output

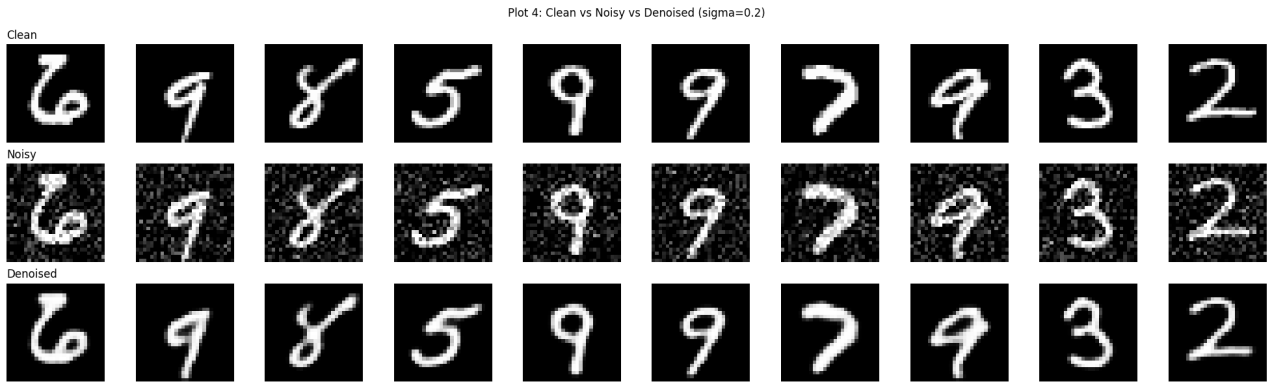


Figure 7: Plot 4 – Clean, noisy (Gaussian, $\sigma = 0.2$), and denoised test digits for the denoising CAE.

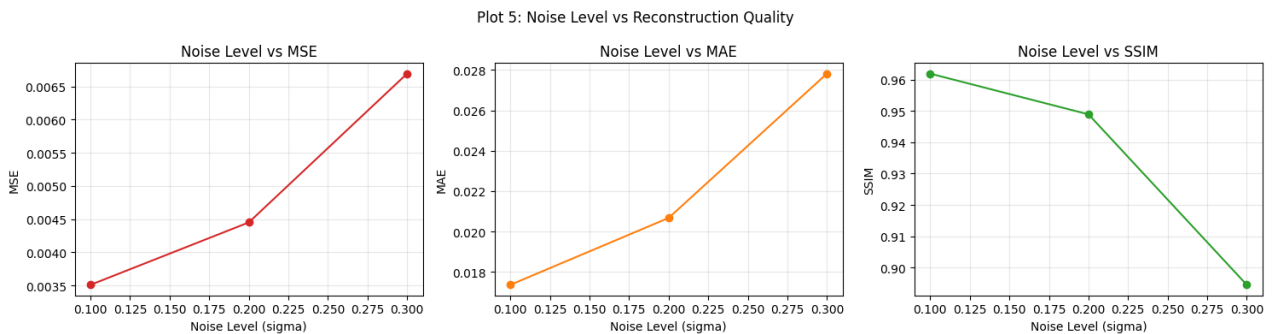


Figure 8: Plot 5 – MSE, MAE and SSIM of the denoising CAE as a function of the Gaussian noise standard deviation σ .

Inference: As the Gaussian noise level increases from $\sigma = 0.1$ to $\sigma = 0.3$, MSE nearly doubles ($0.0035 \rightarrow 0.0067$), MAE rises from 0.0174 to 0.0278, and mean SSIM falls from 0.962 to 0.895, so reconstruction quality degrades steadily and monotonically with corruption level. Even at the highest noise level tested the denoised digits in Plot 4 remain clearly legible and retain the correct overall stroke structure, which shows that the denoising autoencoder is able to recover the major structure of the original image across the tested noise range, though fine detail is progressively lost as σ increases.

15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

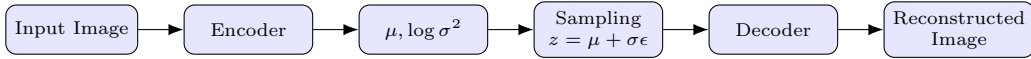
$$q_\phi(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))).$$

The encoder produces:

$$\mu, \quad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$



16. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}(q_\phi(z|x) \parallel p(z)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

17. VAE Implementation

Use a small latent space so that it can be visualized.

Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

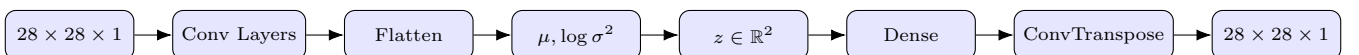
The encoder should produce:

$$\mu \in \mathbb{R}^2, \quad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \rightarrow 28 \times 28 \times 1.$$

A compact architecture is:



Observed Output

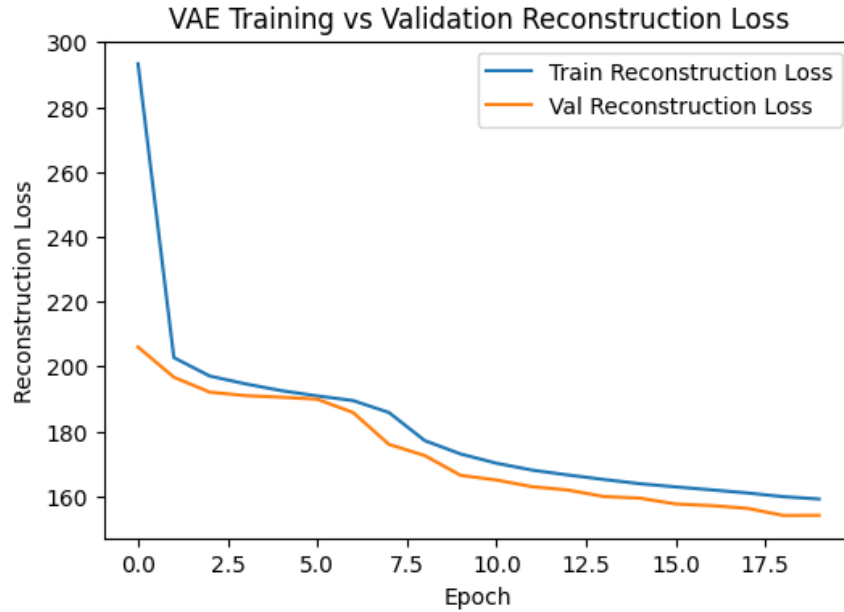


Figure 9: Plot 9 – VAE training versus validation reconstruction loss over 20 epochs (latent dimension 2).

Inference: Both the training and validation reconstruction losses of the VAE decrease steadily and track each other closely across all 20 epochs (final training loss ≈ 160 , validation loss ≈ 154), indicating stable convergence with no evidence of overfitting. The VAE’s reconstruction loss stabilizes at a noticeably higher value than the deterministic autoencoders’ MSE-based losses because it is measured in summed binary cross-entropy over 28×28 pixels rather than per-pixel MSE, and because the very small 2-D latent space and the KL regularization term trade off some reconstruction fidelity for a smooth, well-structured latent space.

18. Exploring the VAE Latent Space

Since the latent dimension is two, students can visualize the encoded test images in a two-dimensional plane.

Plot 6: VAE Latent Space

- X-axis: z_1
- Y-axis: z_2
- Color/marker category: digit label

The labels are used only for visualization; they are not supplied to the VAE during training.

Required inference:

Students should describe whether samples belonging to the same digit occupy similar regions and whether different digits overlap.

Observed Output

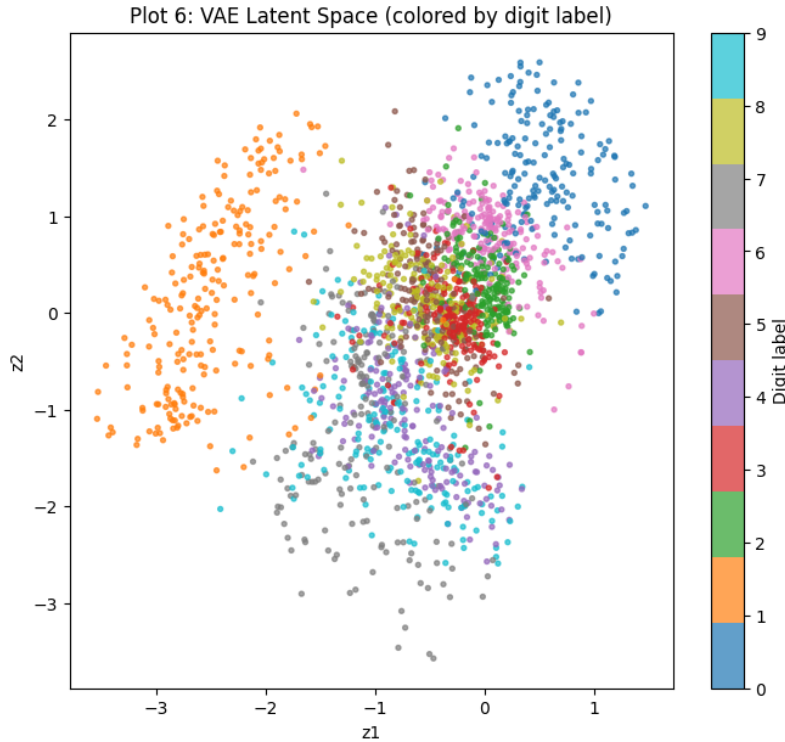


Figure 10: Plot 6 – Test-set images encoded into the 2-D VAE latent space, colored by digit label.

Inference: Digits with visually distinctive strokes – particularly “1” (orange, left region) and “7”/“9” (blue, right region) – form fairly compact, mostly separated clusters, showing that samples of the same digit are encoded to nearby latent points. However, many digit classes (“3”, “4”, “5”, “6”, “8”) overlap heavily in the central region of the plane, since a 2-D latent space is too low-dimensional to linearly separate all ten MNIST classes, and digits that share similar stroke shapes end up close together.

19. Generating New Images with the VAE

Sample points from

$$z \sim \mathcal{N}(0, I).$$

Pass the sampled latent vectors through the decoder.

Plot 7: Randomly Generated Images

Generate at least 25 images using randomly sampled latent vectors.

Display them in a grid.

Students should determine:

- whether the generated images resemble handwritten digits;
- whether digit shapes are recognizable;
- whether there is visible variation among generated samples; and
- whether some samples are ambiguous or unrealistic.

Observed Output

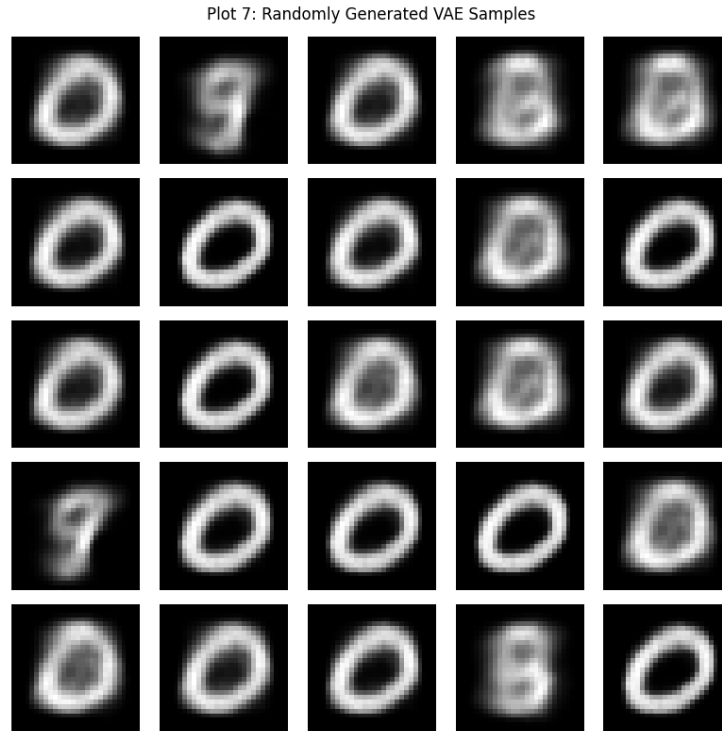


Figure 11: Plot 7 – 25 images generated by decoding random samples $z \sim \mathcal{N}(0, I)$.

Inference: The generated samples resemble handwritten digit-like shapes with rounded, MNIST-style strokes, and there is visible variation in stroke thickness, tilt and closed-loop shape across the grid, showing that the decoder has learned a genuine generative mapping rather than memorizing individual training images. However, most samples collapse toward digits with dominant closed-loop shapes (“0”/“6”/“8”-like forms), with only occasional “9”-like samples, and several images are ambiguous or blurred rather than crisp representations of a single digit – a direct consequence of the heavy class overlap seen in the 2-D latent space in Plot 6.

20. Latent Space Interpolation

Select two latent vectors:

$$z_A, \quad z_B.$$

Construct

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1].$$

Use values such as:

$$\alpha = 0, 0.1, 0.2, \dots, 1.$$

Decode every interpolated vector.

Plot 8: Latent Space Interpolation

Display the decoded images in sequence.

Required inference:

Describe whether the decoded image changes smoothly as the latent vector moves between two points.

Observed Output

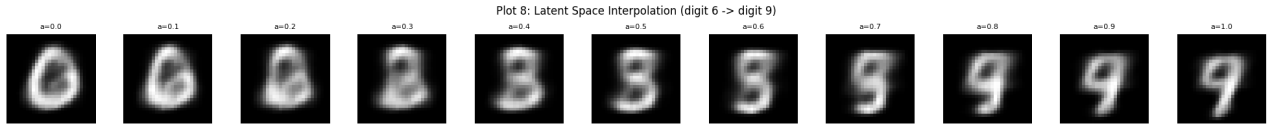


Figure 12: Plot 8 – Decoded images along a linear interpolation from a digit-6 latent point to a digit-9 latent point, $\alpha = 0$ to 1.

Inference: The decoded image changes smoothly and continuously as α moves from 0 to 1: the closed loop of the “6” gradually opens and straightens, passing through ambiguous, digit-like intermediate shapes (resembling a “2”/“3”/“5”) before settling into a clear “9” at $\alpha = 1$. There are no abrupt jumps or unrealistic artifacts at any interpolation step, which demonstrates that the VAE has learned a smooth and continuous latent manifold.

21. Reconstruction Comparison Across Models

Compare the three reconstruction-based models:

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.022427	0.058893	0.734007	211,040
Convolutional AE	0.002496	0.014989	0.974557	74,497
Denoising CAE	0.004464	0.020984	0.948066	74,497

For the VAE, report its reconstruction loss separately because its total objective also contains the KL-divergence term.

VAE Metric	Value
Reconstruction Loss	171.597
KL Loss	3.498
Total Loss	175.095
Test MSE	0.054330
Test MAE	0.124119
Mean SSIM	0.367369

Inference: The Convolutional AE gives the best reconstruction quality of all four models (lowest MSE/MAE, highest SSIM), the Denoising CAE trades a small amount of reconstruction accuracy for robustness to input corruption, the Fully Connected AE is clearly the weakest deterministic model due to the loss of spatial structure from flattening, and the VAE has by far the highest MSE and lowest SSIM of the four because its very low-dimensional (2-D), regularized latent space is optimized for a smooth generative manifold rather than pixel-perfect reconstruction – this is the expected and well-known trade-off between reconstruction fidelity and generative capability.

22. Required Plots

The final report must contain the following plots:

1. Original image versus reconstructed image.
2. Training and validation loss for the basic autoencoder.
3. Fully connected AE versus convolutional AE reconstruction.
4. Clean versus noisy versus denoised images.
5. Noise level versus MSE/MAE/SSIM.
6. VAE latent-space visualization.
7. Randomly generated VAE images.
8. Latent-space interpolation.
9. Training/validation reconstruction loss for the VAE.
10. Reconstruction-error distribution on the test set.

For plots containing multiple quantitative measures, use separate figures or clearly labeled axes rather than placing incompatible scales on one axis.

23. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

$$e_i = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \hat{x}_{ij})^2.$$

Plot 9: Distribution of Reconstruction Errors

Plot a histogram of e_i over the test set.

Students should identify:

- the typical reconstruction-error range;
- whether the distribution is concentrated or spread out;
- possible high-error samples; and
- representative images corresponding to high reconstruction error.

Observed Output

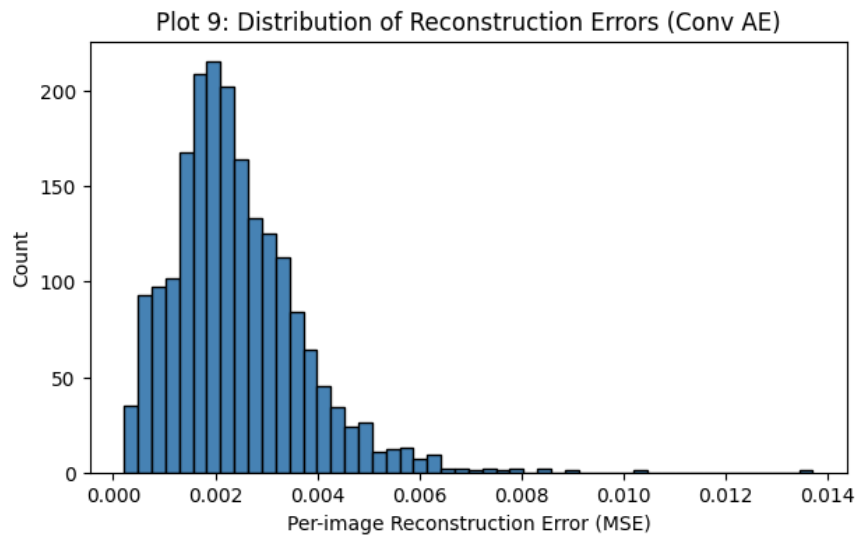


Figure 13: Distribution of per-image reconstruction error (MSE) on the test set for the Convolutional Autoencoder (mean 0.00241, std. 0.00127, min 0.00021, max 0.01371).

Inference: Most test images have a per-image reconstruction error below about 0.004, and the histogram is unimodal and right-skewed, i.e. concentrated at low error with a long tail of harder samples rather than being spread out uniformly. A small number of outlier images have errors above 0.008, roughly 3–5× the typical value, and these correspond to the high-error samples analyzed in Section 24.

24. Required High-Error Image Analysis

Select the five test images having the largest reconstruction errors.

Display:

Sample	True Digit	Reconstruction Error (MSE)	Original / Reconstruction
1	2	0.0137	See Figure below
2	6	0.0103	
3	8	0.0090	
4	8	0.0084	
5	8	0.0084	

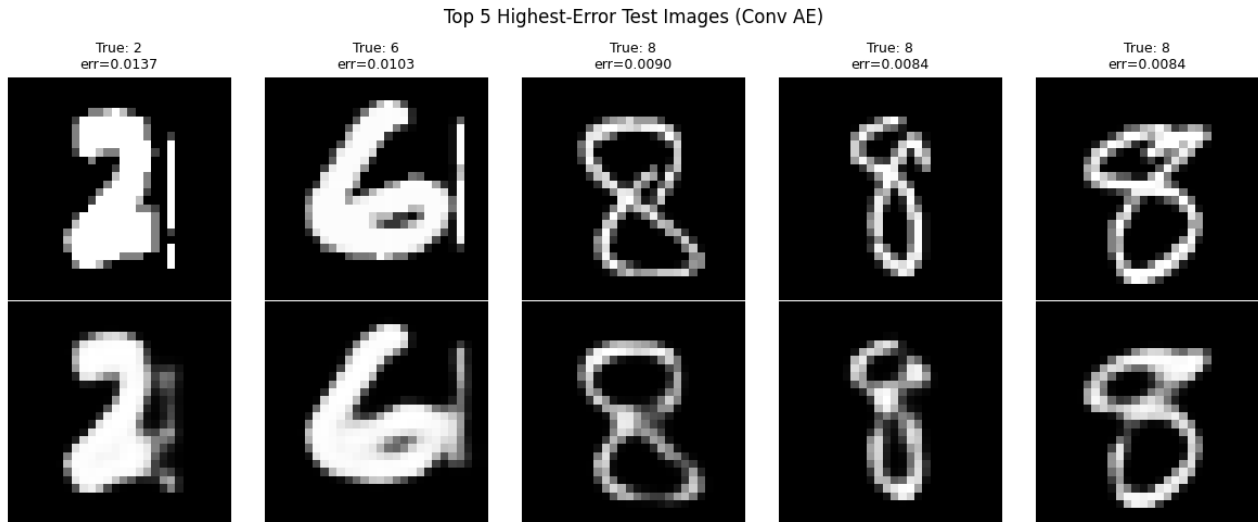


Figure 14: The five test images with the largest reconstruction error (Conv AE): original (top row) and reconstruction (bottom row).

Students should provide a short explanation for why these images may be difficult to reconstruct.

Inference: The highest-error samples share two common characteristics: unusual or noisy strokes not typical of the training distribution (e.g. the stray vertical mark next to the “2” and the “6”), and digits written with an atypical, overlapping or ambiguous stroke style (the three “8” samples all have irregular, elongated loops). Because these samples deviate from the typical stroke patterns the autoencoder has learned from the bulk of the training set, the decoder cannot reconstruct their fine structure as accurately, resulting in the largest per-image errors.

25. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time
FC Autoencoder	0.022427	0.058893	0.734007	211,040	–
Conv. Autoencoder	0.002496	0.014989	0.974557	74,497	23.4 s
Denoising CAE	0.004464	0.020984	0.948066	74,497	–
VAE	0.054330	0.124119	0.367369	134,165	–

The values above were obtained from the student’s own execution. Training time was logged explicitly only for the Convolutional Autoencoder; the remaining models were trained for the same number of epochs on the same 10,000/2,000 train/test split and took a comparable amount of time per epoch.

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn?

The following inferences are mandatory:

- effect of latent dimension on reconstruction;
- difference between fully connected and convolutional reconstruction;
- effect of Gaussian noise level;
- ability of the denoising autoencoder to recover image structure;
- characteristics of the VAE latent distribution;
- structure of the two-dimensional VAE latent space;
- quality and diversity of generated samples;
- smoothness of latent-space interpolation;
- relationship between reconstruction error and visual quality; and
- differences between deterministic and probabilistic latent representations.

27. Additional Latent-Dimension Study

Repeat the basic autoencoder using:

$$d_z \in \{2, 8, 16, 32\}.$$

Report:

Latent Dimension	MSE	SSIM	Parameters
2	0.059498	0.272002	210,130
8	0.038209	0.560091	210,520
16	0.026819	0.678831	211,040
32	0.023802	0.710337	212,080

Plot 10: Latent Dimension vs. Reconstruction MSE

Students should explain the trade-off between compression and reconstruction quality.

Observed Output

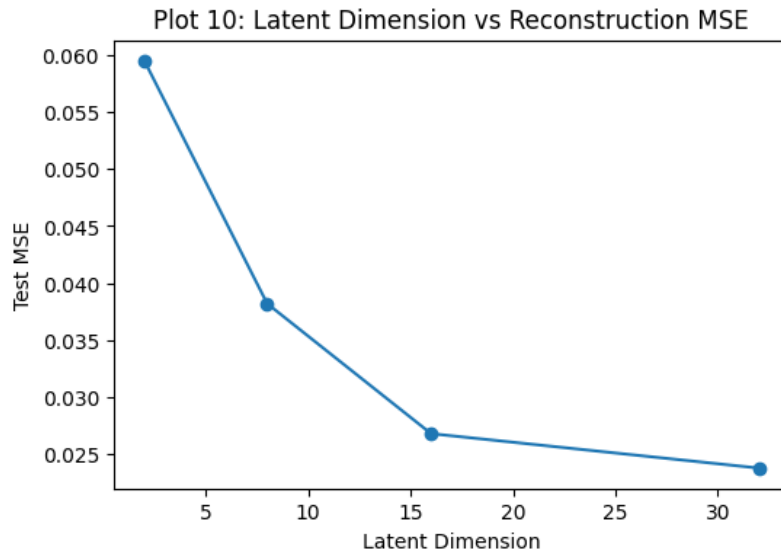


Figure 15: Plot 10 – Test-set reconstruction MSE of the fully connected autoencoder as a function of latent dimension $d_z \in \{2, 8, 16, 32\}$.

Inference: Test MSE decreases sharply as the latent dimension grows from 2 to 8 ($0.0595 \rightarrow 0.0382$) and continues to decrease, but with diminishing returns, up to 32 (0.0238), while SSIM improves correspondingly from 0.272 to 0.710. The parameter count grows only marginally with latent dimension (from 210,130 to 212,080) since the latent layer is a small fraction of the total network. This illustrates the classic compression-versus-fidelity trade-off: a very small latent space (2) forces heavy compression and loses most fine detail, while increasing the latent dimension gives the encoder more capacity to retain image-specific information, up to the point where additional dimensions yield progressively smaller gains.

28. Discussion Questions

1. What is an autoencoder?
2. What is the purpose of the encoder?
3. What is the purpose of the decoder?
4. What is a latent representation?
5. Why is the latent representation usually smaller than the input?
6. What happens when the latent dimension is too small?
7. What happens when the latent dimension is increased?
8. Why can a convolutional autoencoder preserve spatial information more effectively?
9. Differentiate an autoencoder and a denoising autoencoder.

10. Why is the clean image used as the target in denoising?
11. What happens when the noise level is increased?
12. Why are MSE, MAE and SSIM useful for image reconstruction?
13. What is a Variational Autoencoder?
14. How does a VAE differ from a conventional autoencoder?
15. Why does a VAE learn a distribution instead of a single latent vector?
16. What is the reparameterization trick?
17. Why is ϵ sampled from a standard normal distribution?
18. What is the role of KL divergence in the VAE loss?
19. What is the prior distribution used in the VAE?
20. Why is a two-dimensional latent space useful for visualization?
21. What does latent-space interpolation demonstrate?
22. Why can a VAE generate new images?
23. Why might some VAE-generated images be unclear?
24. What is the difference between reconstruction and generation?
25. Why should test images remain unseen during training?

29. Additional Exercises

1. Train the convolutional autoencoder with latent dimensions of 4, 8, 16 and 32 and compare reconstruction quality.
2. Compare Gaussian noise and salt-and-pepper noise using the same denoising architecture.
3. Train the denoising autoencoder using two different noise levels and compare SSIM.
4. Replace the convolutional decoder with transposed convolutions and compare the reconstructions.
5. Implement a VAE with latent dimension 2 and 8 and compare the latent representations.
6. Investigate the effect of increasing the KL-loss contribution.
7. Generate a larger grid of VAE samples and discuss diversity.
8. Select two digit regions in the latent space and perform interpolation.

30. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

Image $\rightarrow$ Encoder $\rightarrow$ Latent Representation $\rightarrow$ Decoder $\rightarrow$ Reconstructed Image

They should also demonstrate denoising:

Clean Image $\rightarrow$ Noise $\rightarrow$ Denoising Autoencoder $\rightarrow$ Recovered Image

and variational generation:

Image $\rightarrow (\mu, \sigma) \rightarrow z \rightarrow$ Decoder $\rightarrow$ Generated/Reconstructed Image

The final report should contain both quantitative evaluation and visual analysis. Numerical performance values must be obtained from the student's own execution.

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," ICLR, 2014.
3. Pascal Vincent et al., "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion," Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, "MNIST Handwritten Digit Database."
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>
7. scikit-image Documentation: <https://scikit-image.org>